

# Laboratório 3 - Dados relacionais e operações do tipo join

Pedro Tilhe

2026-08-30

**Carregue os pacotes relevantes para este Laboratório**

```
library(readr)
library(dplyr)
library(leaflet)
```

## Atrasos de vôos

### 1. Importação dos três arquivos

Os arquivos `airlines.csv` e `airports.csv` estão na pasta `dados/`. O arquivo `flights.csv` é grande (~590 MB) e está compactado em `flights.csv.zip`; ele é descompactado apenas se ainda não existir.

```
airlines <- read_csv("dados/airlines.csv")

airports <- read_csv(
  "dados/airports.csv",
  col_select = c(IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE)
)

airlines
```

```
# A tibble: 14 x 2
  IATA_CODE AIRLINE
  <chr>      <chr>
```

```

1 UA      United Air Lines Inc.
2 AA      American Airlines Inc.
3 US      US Airways Inc.
4 F9      Frontier Airlines Inc.
5 B6      JetBlue Airways
6 OO      Skywest Airlines Inc.
7 AS      Alaska Airlines Inc.
8 NK      Spirit Air Lines
9 WN      Southwest Airlines Co.
10 DL     Delta Air Lines Inc.
11 EV     Atlantic Southeast Airlines
12 HA     Hawaiian Airlines Inc.
13 MQ     American Eagle Airlines Inc.
14 VX     Virgin America

```

## airports

```

# A tibble: 322 x 5
  IATA_CODE CITY      STATE LATITUDE LONGITUDE
  <chr>      <chr>      <chr>    <dbl>    <dbl>
1 ABE      Allentown    PA        40.7     -75.4
2 ABI      Abilene      TX        32.4     -99.7
3 ABQ      Albuquerque  NM        35.0    -107.
4 ABR      Aberdeen     SD        45.4     -98.4
5 ABY      Albany       GA        31.5     -84.2
6 ACK      Nantucket    MA        41.3     -70.1
7 ACT      Waco         TX        31.6     -97.2
8 ACV      Arcata/Eureka CA        41.0    -124.
9 ACY      Atlantic City NJ        39.5     -74.6
10 ADK     Adak         AK        51.9    -177.
# i 312 more rows

```

Para `flights.csv`, como o arquivo é grande, ele é lido em pedaços (“chunks”) de 1 milhão de linhas por vez com `read_csv_chunked()`. Para cada pedaço: mantemos apenas `DESTINATION_AIRPORT` e `ARRIVAL_DELAY`, removemos vôos cujo aeroporto de destino começa com “1” (esses são códigos numéricos de aeroporto sem `IATA_CODE` correspondente, que não aparecem em `airports.csv`) e removemos registros com alguma coluna faltante. Em seguida calculamos, para cada aeroporto, a soma dos atrasos e a contagem de vôos — essas duas quantidades são as “estatísticas suficientes” para depois obter a média (basta dividir a soma acumulada pela contagem acumulada), sem precisar manter o arquivo inteiro em memória.

```

if (!file.exists("flights.csv")) {
  unzip("flights.csv.zip")
}

estatisticas_parciais <- list()

processa_pedaco <- function(pedaco, pos) {
  pedaco <- pedaco %>%
    filter(!startsWith(DESTINATION_AIRPORT, "1")) %>%
    na.omit()

  resumo <- pedaco %>%
    group_by(DESTINATION_AIRPORT) %>%
    summarise(soma_atraso = sum(ARRIVAL_DELAY), n_voos = n(), .groups = "drop")

  estatisticas_parciais[[length(estatisticas_parciais) + 1]] <- resumo
}

read_csv_chunked(
  "flights.csv",
  callback = SideEffectChunkCallback$new(processa_pedaco),
  chunk_size = 1e6,
  col_types = cols_only(
    DESTINATION_AIRPORT = col_character(),
    ARRIVAL_DELAY = col_double()
  )
)

```

NULL

```

# Combina as estatísticas suficientes de todos os pedaços
flights <- bind_rows(estatisticas_parciais) %>%
  group_by(DESTINATION_AIRPORT) %>%
  summarise(soma_atraso = sum(soma_atraso), n_voos = sum(n_voos), .groups = "drop") %>%
  mutate(atraso_medio = soma_atraso / n_voos)

flights

```

```

# A tibble: 322 x 4
  DESTINATION_AIRPORT soma_atraso n_voos atraso_medio
  <chr>                <dbl>   <int>         <dbl>

```

```

1 ABE          12874   2220      5.80
2 ABI           9077   2224      4.08
3 ABQ        106415  18953      5.61
4 ABR          -2227    657     -3.39
5 ABY           7501    864      8.68
6 ACK          2393    487      4.91
7 ACT        16392   1541     10.6
8 ACV        12259   1267      9.68
9 ACY        41744   3530     11.8
10 ADK          -284     89     -3.19
# i 312 more rows

```

**Conferência:** com os dados completos (2015, ~5,8 milhões de vôos), depois de remover os ~307 códigos numéricos e os registros incompletos, restam 5.231.130 vôos válidos e exatamente **322 aeroportos de destino distintos** — o mesmo número de linhas de `airports.csv`.

## 2. *Join* de flights com airports

- a. A chave em `flights` é `DESTINATION_AIRPORT`.
- b. A chave em `airports` é `IATA_CODE`.
- c. Aeroportos que aparecem em `flights` mas estão ausentes em `airports`:

```
anti_join(flights, airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))
```

```

# A tibble: 0 x 4
# i 4 variables: DESTINATION_AIRPORT <chr>, soma_atraso <dbl>, n_voos <int>,
#   atraso_medio <dbl>

```

Não há nenhum (o resultado é uma tabela vazia). Isso porque o passo 1.c já havia removido os códigos numéricos que começam com “1” — são justamente esses códigos que não têm correspondência em `airports.csv`. Depois dessa limpeza, todo aeroporto de destino de `flights` tem uma linha correspondente em `airports`.

- d. Como queremos manter todos os registros de `flights` (mesmo que, em outra situação, algum não tivesse par em `airports`), a operação apropriada é o `left_join`, usando `DESTINATION_AIRPORT` (`flights`) e `IATA_CODE` (`airports`) como chaves:

```

flights <- left_join(flights, airports,
                    by = c("DESTINATION_AIRPORT" = "IATA_CODE"))
flights

```

```
# A tibble: 322 x 8
  DESTINATION_AIRPORT soma_atraso n_voos atraso_medio CITY STATE LATITUDE
  <chr> <dbl> <int> <dbl> <chr> <chr> <dbl>
1 ABE 12874 2220 5.80 Allentown PA 40.7
2 ABI 9077 2224 4.08 Abilene TX 32.4
3 ABQ 106415 18953 5.61 Albuquerque NM 35.0
4 ABR -2227 657 -3.39 Aberdeen SD 45.4
5 ABY 7501 864 8.68 Albany GA 31.5
6 ACK 2393 487 4.91 Nantucket MA 41.3
7 ACT 16392 1541 10.6 Waco TX 31.6
8 ACV 12259 1267 9.68 Arcata/Eu~ CA 41.0
9 ACY 41744 3530 11.8 Atlantic ~ NJ 39.5
10 ADK -284 89 -3.19 Adak AK 51.9
# i 312 more rows
# i 1 more variable: LONGITUDE <dbl>
```

e. A tabela resultante já foi armazenada de volta no objeto `flights`.

### 3. Quantidade de aeroportos por estado

```
aerportos_por_estado <- airports %>%
  group_by(STATE) %>%
  summarise(n_aerportos = n(), .groups = "drop") %>%
  arrange(desc(n_aerportos))

aerportos_por_estado
```

```
# A tibble: 54 x 2
  STATE n_aerportos
  <chr> <int>
1 TX 24
2 CA 22
3 AK 19
4 FL 17
5 MI 15
6 NY 14
7 CO 10
8 MN 8
9 MT 8
10 NC 8
# i 44 more rows
```

Os estados (e territórios) com mais aeroportos no arquivo são Texas (24), Califórnia (22) e Alasca (19).

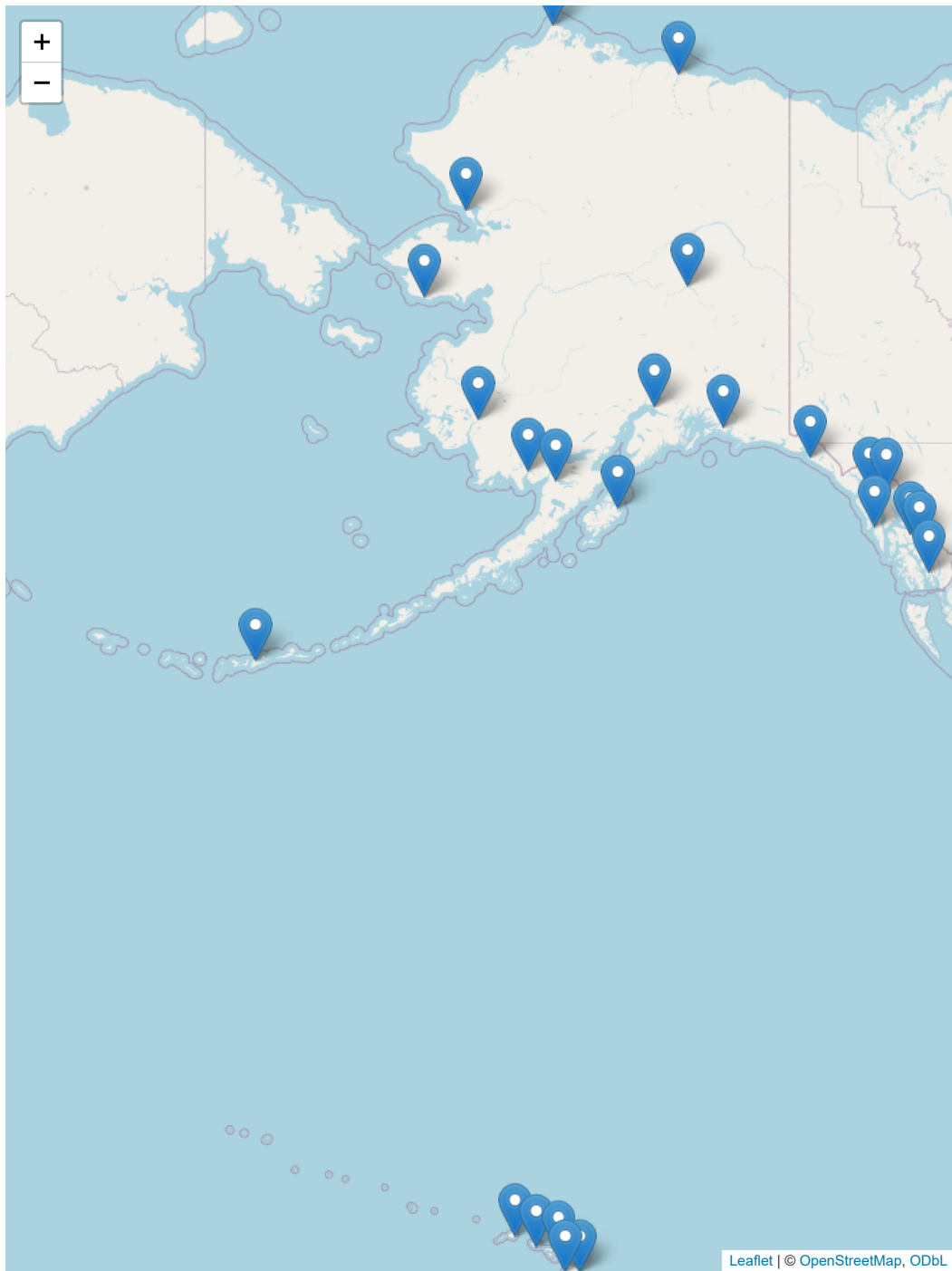
#### 4. Mapa dos atrasos por aeroporto

```
# a. pacote leaflet já carregado no início do documento

# b. e c. mapa básico armazenado em `this`
mapa_dados <- flights %>% filter(!is.na(LATITUDE), !is.na(LONGITUDE))

this <- leaflet(mapa_dados) %>%
  addTiles() %>%
  addMarkers(
    lng = ~LONGITUDE,
    lat = ~LATITUDE,
    popup = ~paste0(DESTINATION_AIRPORT, ": ", round(atraso_medio, 1), " min")
  )

this
```



```
# d. apenas se estiver usando Jupyter (não necessário no RStudio/Quarto):  
# library(IRdisplay)  
# display(this)
```

**Observação:** 3 aeroportos (ECP, PBG, UST) não têm latitude/longitude em `airports.csv` e por isso foram excluídos do mapa.

### Horário de compilação

```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-30 12:33:50 -03"
```